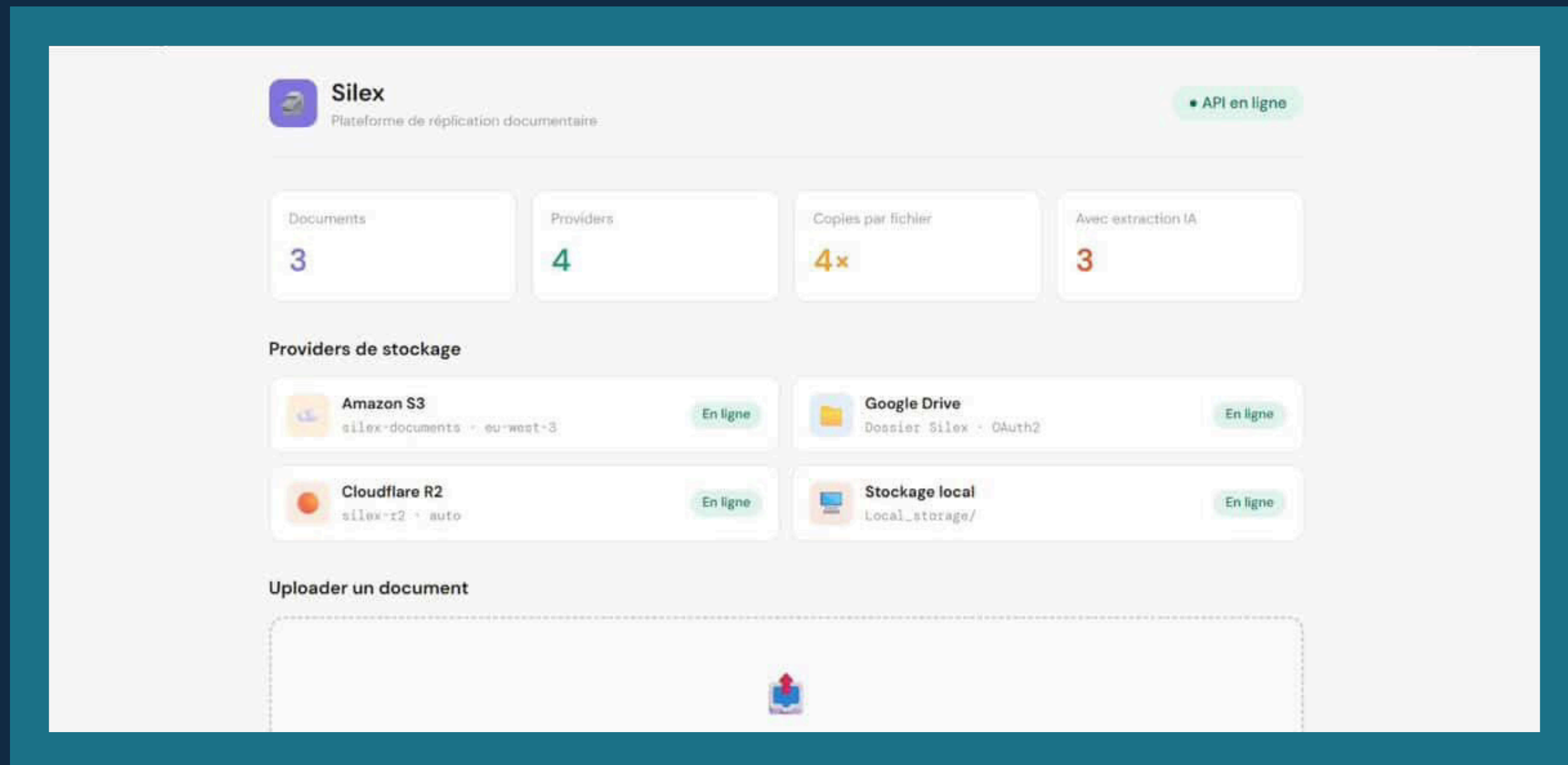


SILEX Plateforme Multi-Cloud



Réalisé par Felix Martinet, Karl Juttin et Kenza Laaziri

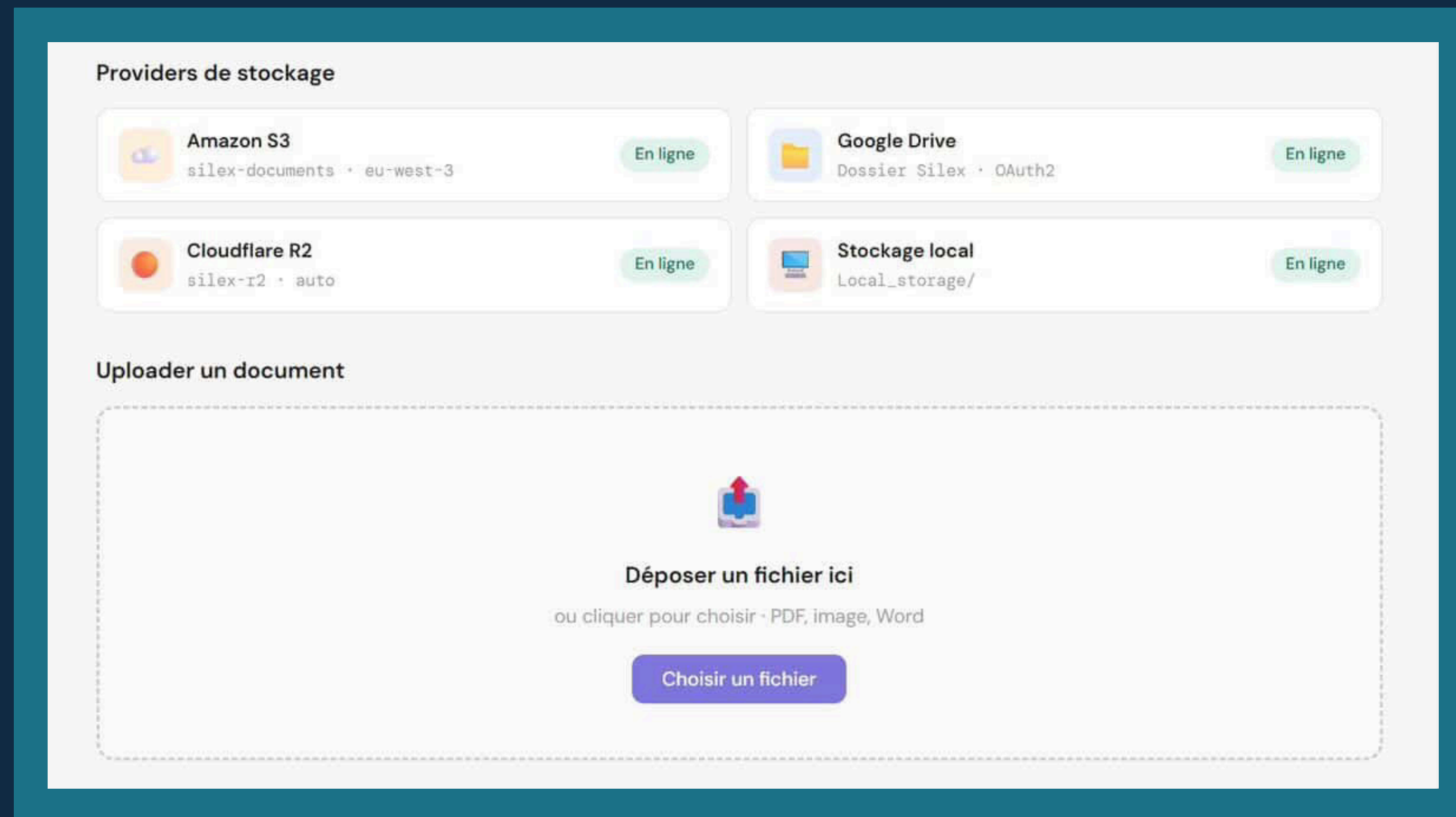
Plan de la présentation

1. PROBLÉMATIQUE DE LA PLATEFORME
2. OBJECTIF DU PROJET
3. ANALYSE DES BESOINS FONCTIONNELS
4. TECHNOLOGIES UTILISÉES POUR SILEX
5. MÉCANISME DE FAN-OUT
6. ARCHITECTURE DE LA SOLUTION

7. ARCHITECTURE DU BACKEND
8. ARCHITECTURE DU FRONTEND
9. ASSISTANT IA CONVERSATIONNEL
10. FONCTIONNALITÉS LIVRÉES ET VALIDÉES
11. LIMITES ET PERSPECTIVES
12. CONCLUSION

Problématiques de la plateforme SILEX

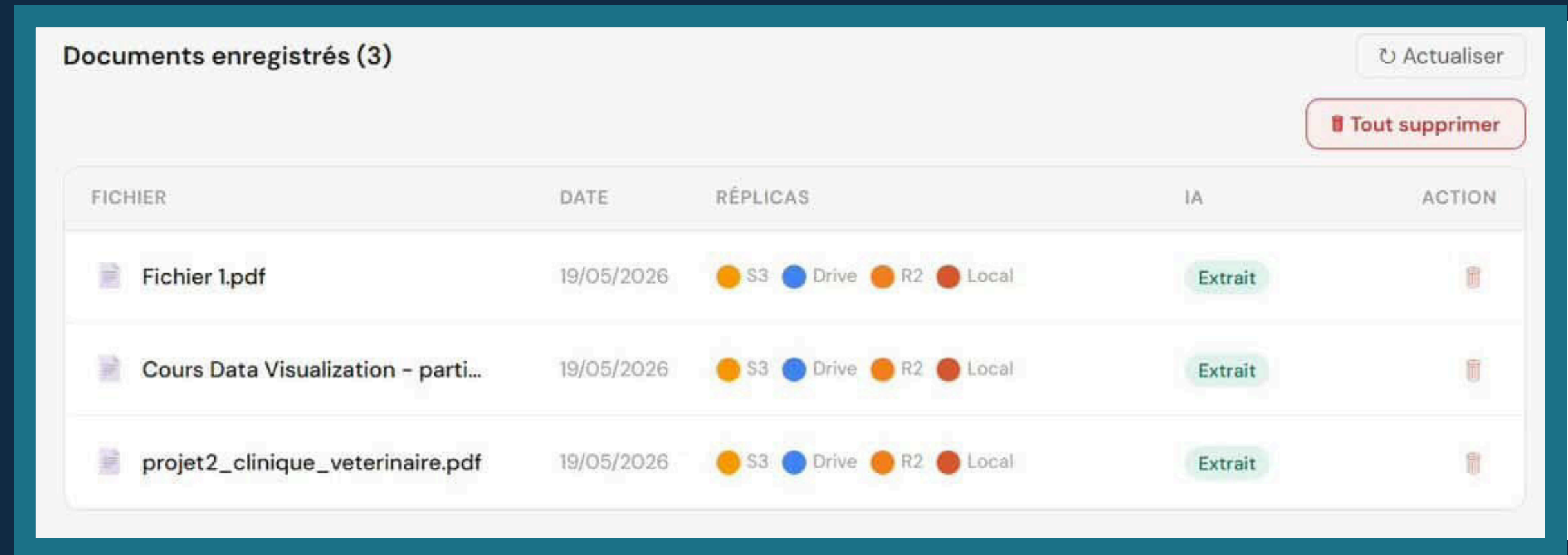
Les fichiers d'une organisation sont aujourd'hui éparpillés sur de multiples sources S3, Google Drive, stockage local et R2 rendant toute recherche unifiée, tout accès centralisé et toute exploitation par l'IA impossible sans une couche d'abstraction commune.



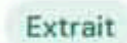
Objectifs du Projet SILEX

Trois objectifs fondamentaux sont établis dans notre projet :

- Ingestion Multi-Source
- Agent IA Documentaire
- Moteur de Réplication



The screenshot displays a web interface titled "Documents enregistrés (3)". It features a table with five columns: FICHIER, DATE, RÉPLICAS, IA, and ACTION. There are three rows of data, each representing a PDF document. Above the table, there are two buttons: "Actualiser" (refresh) and "Tout supprimer" (delete all). The "RÉPLICAS" column shows four status indicators: S3 (yellow), Drive (blue), R2 (orange), and Local (red). The "IA" column shows a green "Extrait" button for each document. The "ACTION" column contains a trash icon for each document.

FICHIER	DATE	RÉPLICAS	IA	ACTION
 Fichier 1.pdf	19/05/2026	 S3  Drive  R2  Local	 Extrait	
 Cours Data Visualization – parti...	19/05/2026	 S3  Drive  R2  Local	 Extrait	
 projet2_clinique_veterinaire.pdf	19/05/2026	 S3  Drive  R2  Local	 Extrait	

Analyse des besoins fonctionnels

5 besoins fonctionnels

- 1 Ingestion multi-source**
Upload de fichiers PDF, images, Word depuis l'interface
- 2 Réplication automatique (Fan-Out)**
Copie simultanée vers S3, Google Drive, R2 et stockage local
- 3 Extraction IA des documents**
Analyse du contenu textuel via PyMuPDF pour l'assistant
- 4 Assistant conversationnel Ody**
ChatBot Ody interrogeant les documents répliqués
- 5 Gestion & suppression coordonnée**
Suppression synchronisée sur l'ensemble des providers

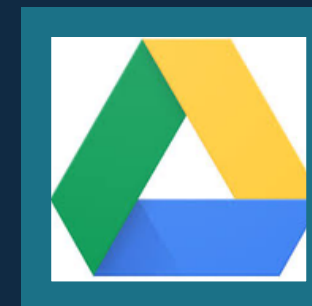
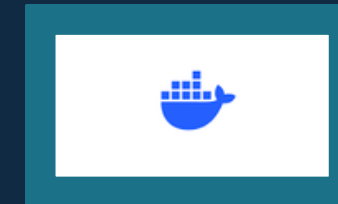
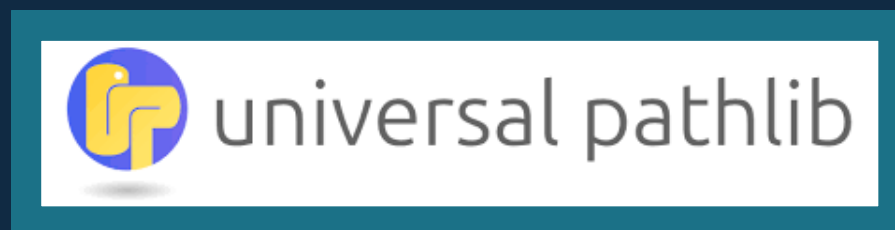
4 exigences non fonctionnelles

- 1 Performance**
Réplication parallèle < 5 s pour un fichier de 10 Mo
- 2 Sécurité**
Aucune clé API committée ; variables d'environnement uniquement
- 3 Maintenabilité**
Architecture modulaire : ajout d'un provider via une seule classe héritée
- 4 Résilience**
Échec partiel toléré ; les providers disponibles continuent la sync

Technologies utilisées pour SILEX

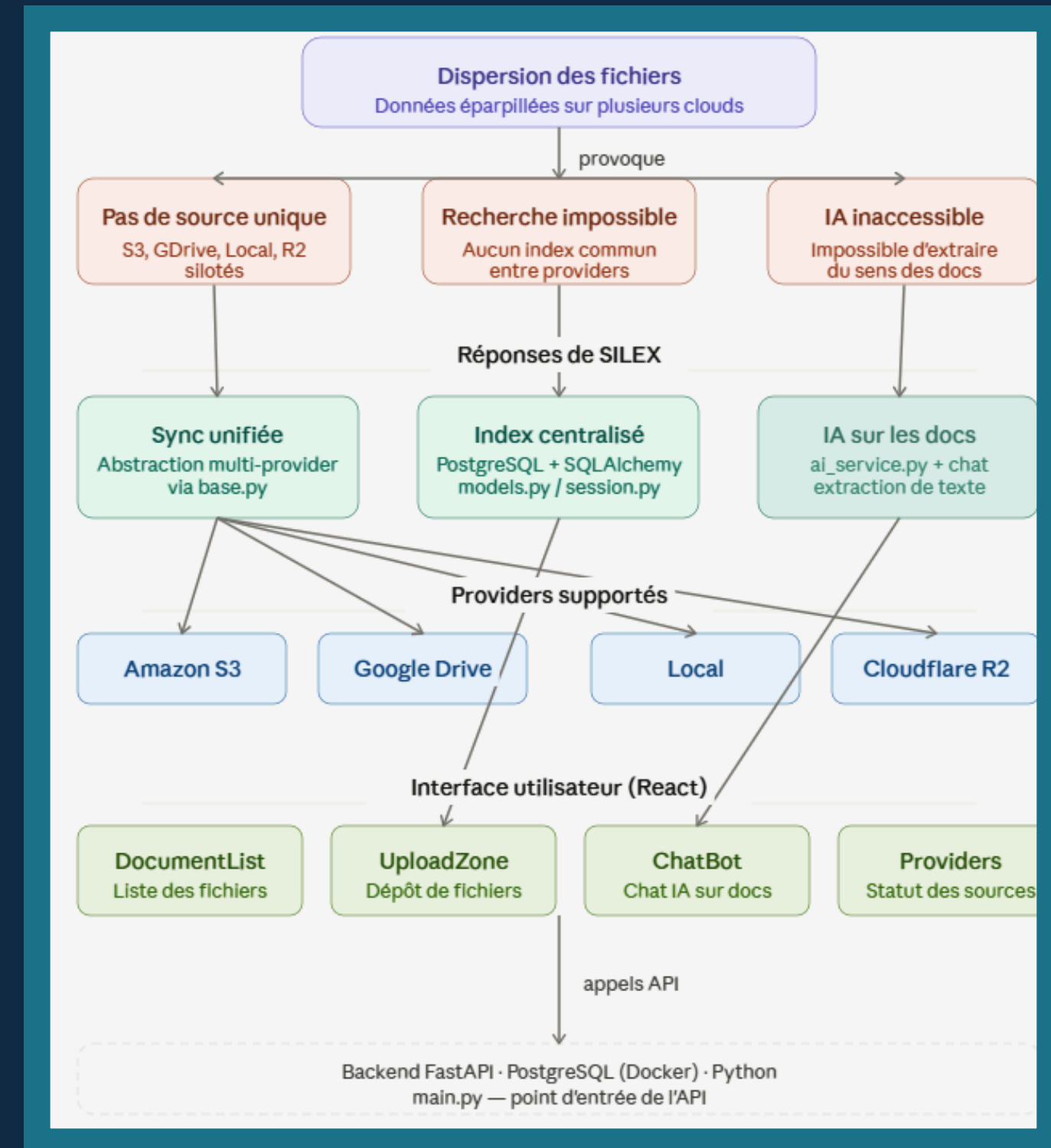
La plateforme SILEX utilise une stack technique, incluant FastAPI pour le backend, PostgreSQL pour la base de données et React pour le frontend, garantissant performance et évolutivité au service de l'utilisateur.

- sqlalchemy
- fastapi
- providers
- anthropic
- google.oauth2
- pathlib
- boto3
- React
- Docker
- GitHub



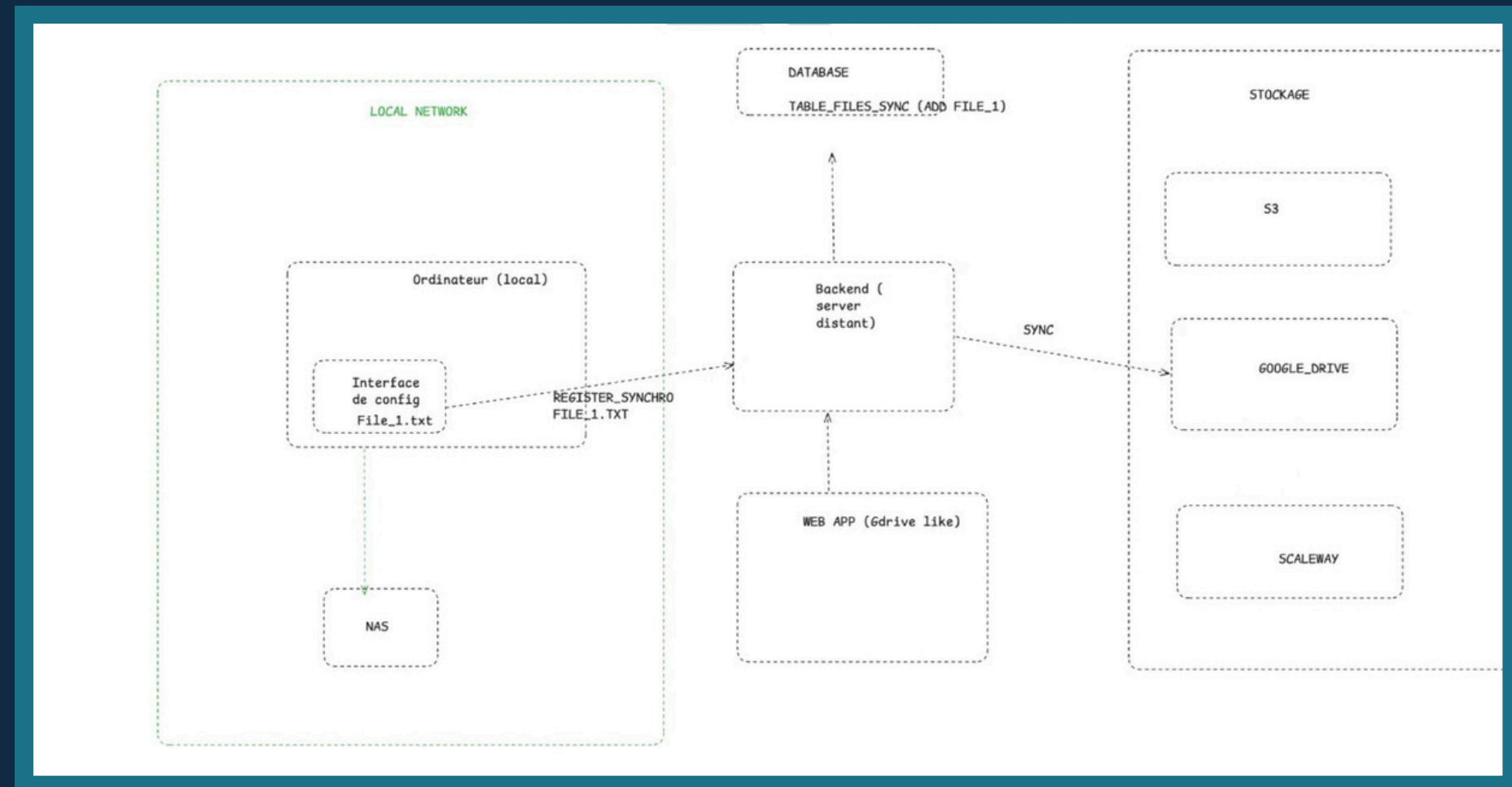
Mécanisme de Fan-Out

Le mécanisme de fan-out assure que chaque document déposé est automatiquement répliqué vers quatre destinations simultanément, garantissant une redondance efficace et une protection contre les pertes de données potentielles.



Architecture de la solution

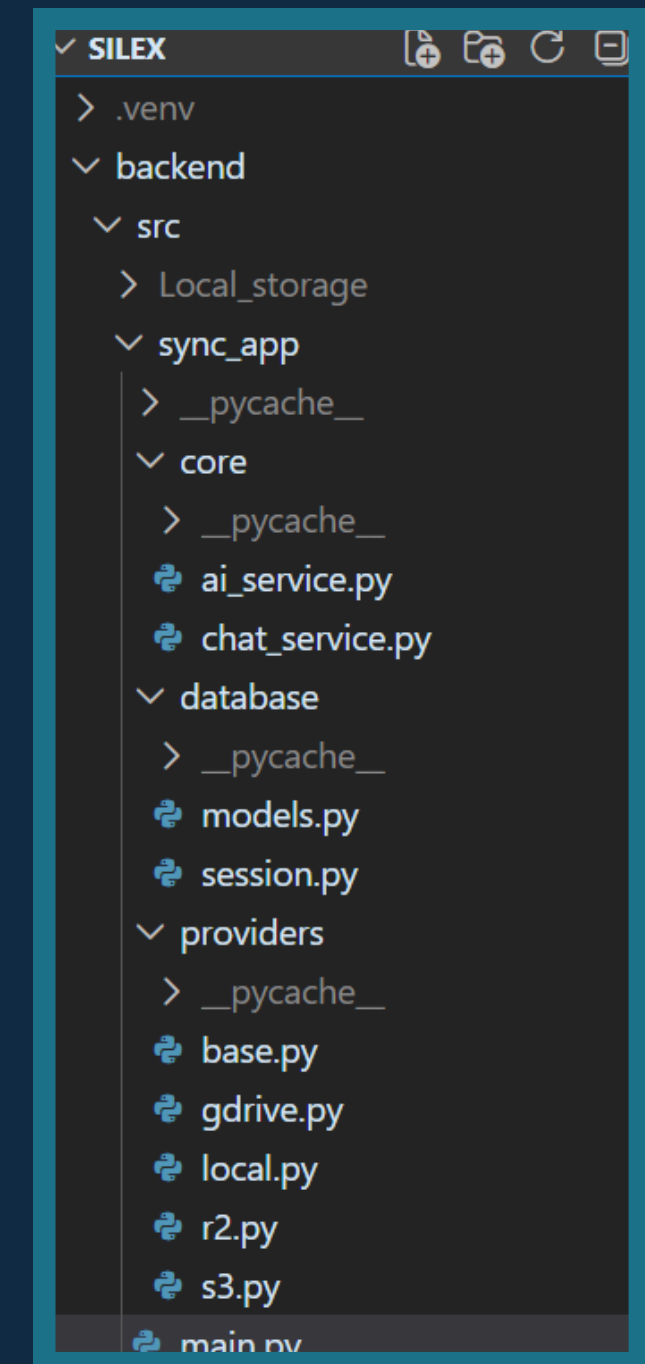
La solution SILEX repose sur une architecture à trois couches, garantissant une communication fluide entre le frontend, le backend et la persistance des données, tout en étant totalement découplée pour une flexibilité optimale.



Architecture du Backend

Le backend est organisé en quatre modules distincts:

- Le dossier providers/ regroupe les quatre implémentations de stockage
- base.py définit la classe abstraite commune.
- s3.py, r2.py, gdrive.py, local.py héritent chacun de cette base.
- Le dossier core/ contient la logique IA avec ai_service.py pour l'extraction PyMuPDF et chat_service.py pour l'assistant Ody.
- Le dossier database/ gère le modèle PostgreSQL via models.py et la connexion SQLAlchemy via session.py.
- main.py est le point d'entrée FastAPI qui orchestre l'ensemble.

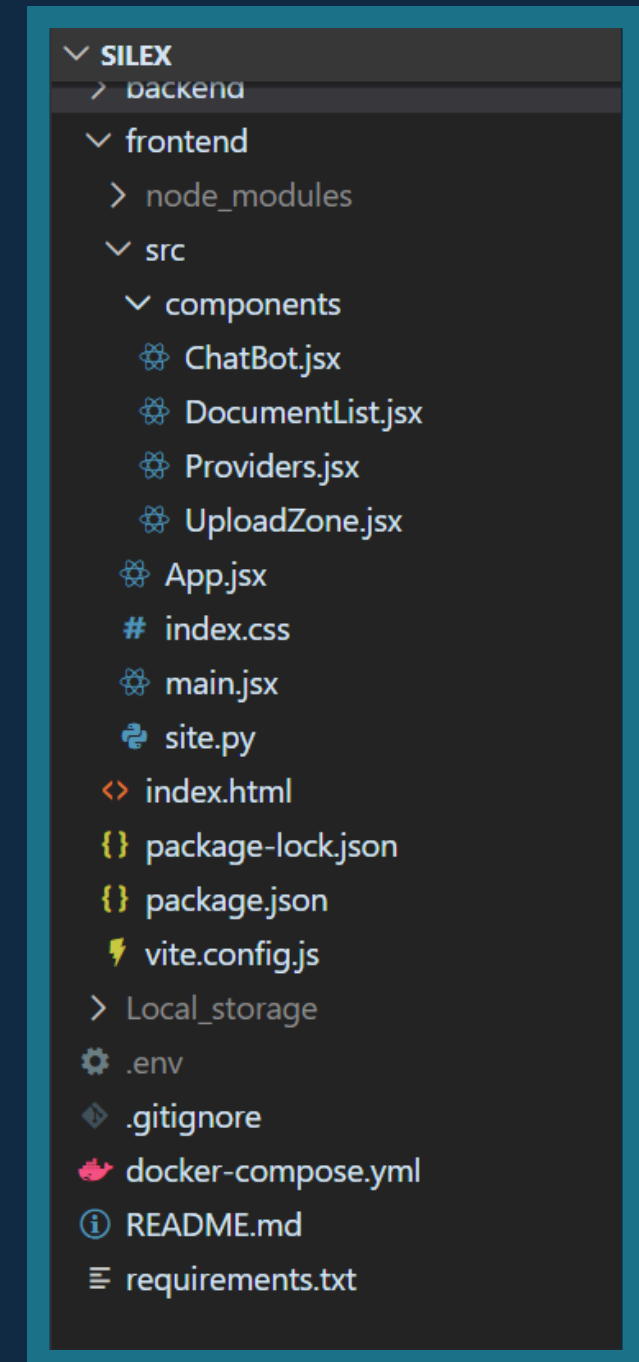


```
Silex/
├── backend/
│   ├── src/
│   │   ├── Local_storage/
│   │   └── sync_app/
│   │       ├── core/
│   │       │   ├── ai_service.py      # Service IA (extraction texte)
│   │       │   └── chat_service.py    # Service de chat
│   │       ├── database/
│   │       │   ├── models.py          # Modèles SQLAlchemy
│   │       │   └── session.py          # Session base de données
│   │       ├── providers/
│   │       │   ├── base.py             # Classe de base provider
│   │       │   ├── gdrive.py           # Provider Google Drive
│   │       │   ├── local.py            # Provider Local
│   │       │   ├── r2.py               # Provider Cloudflare R2
│   │       │   └── s3.py               # Provider S3
│   │       └── main.py                 # Point d'entrée FastAPI
│   ├── client_secrets.json            # Ne pas commiter !
│   └── token.json                    # Ne pas commiter !
```

Architecture du Frontend

Le frontend est organisé en quatre composants React distincts :

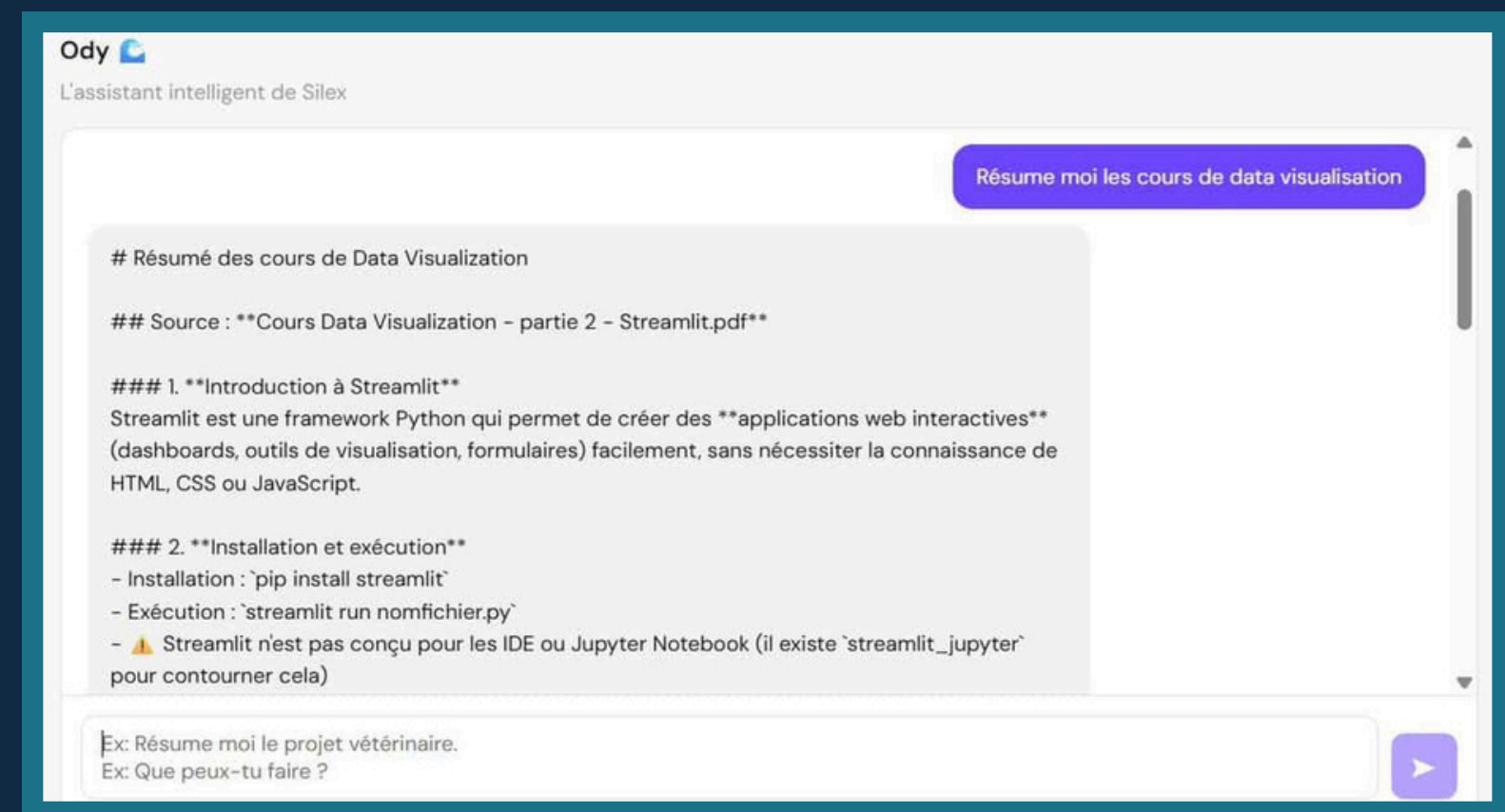
- ChatBot.jsx gère l'interface conversationnelle d'Ody
- DocumentList.jsx affiche la liste des documents répliqués avec leurs statuts
- Providers.jsx affiche l'état des quatre providers en temps réel
- UploadZone.jsx gère le drag & drop
- App.jsx orchestre l'ensemble et maintient l'état global
- La configuration Vite via vite.config.js proxifie les appels vers le backend pour éviter les problèmes CORS



```
|— frontend/
|   |— src/
|   |   |— components/
|   |   |   |— ChatBot.jsx           # Interface chatbot
|   |   |   |— DocumentList.jsx      # Liste des documents
|   |   |   |— Providers.jsx         # Statut des providers
|   |   |   |— UploadZone.jsx        # Zone d'upload
|   |   |— App.jsx                   # Page principale
|   |   |— index.css
|   |   |— main.jsx
|   |   |— site.py
|   |— index.html
|   |— package.json
|   |— package-lock.json
|   |— vite.config.js
|— Local_storage/
|— .env                             # Ne pas commiter !
|— .gitignore
|— docker-compose.yml               # Base de données PostgreSQL
|— requirements.txt                 # Dépendances Python
|— README.md
```

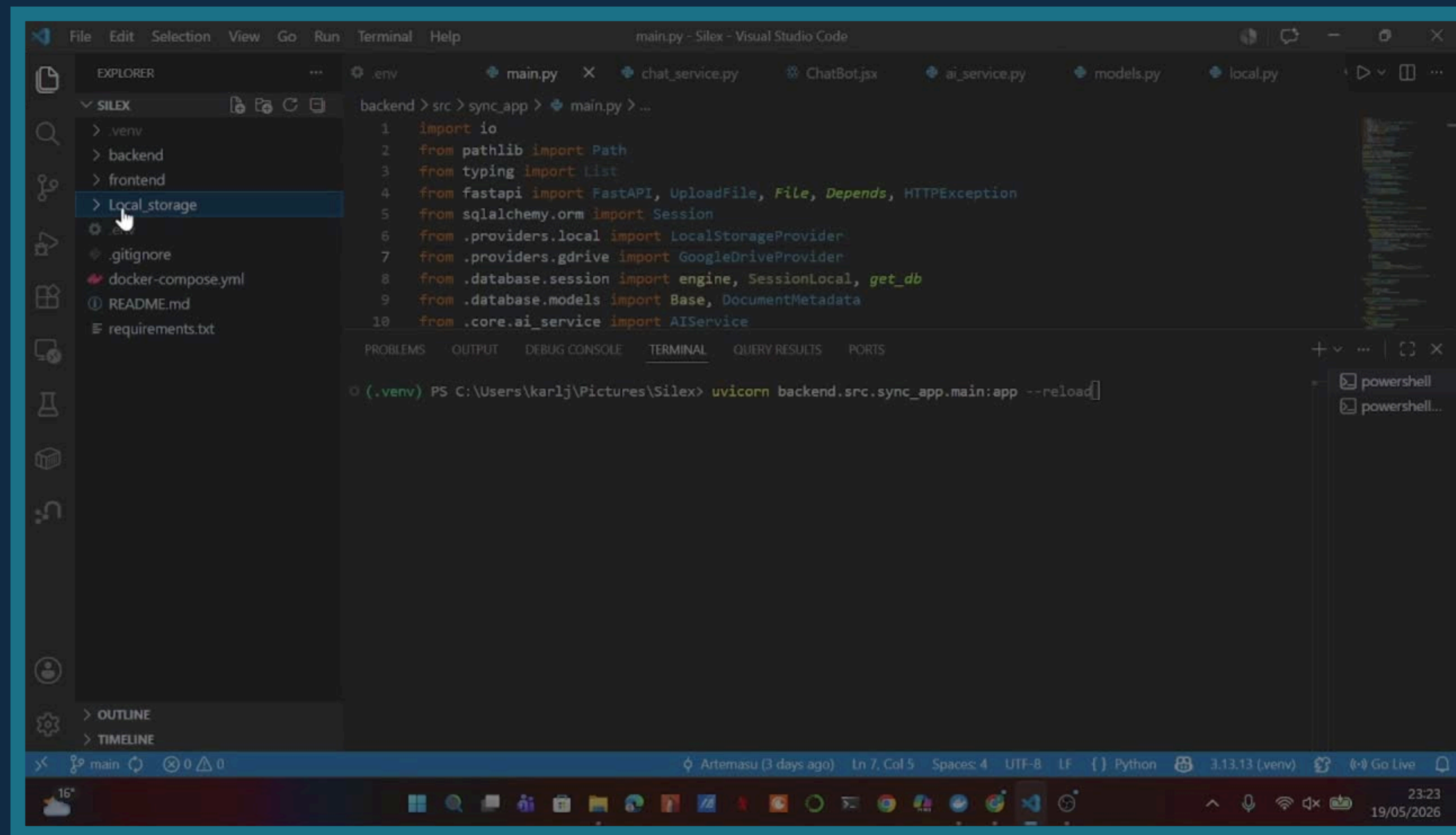

Ody: Assistant IA Conversationnel

Ody est un ChatBot dirigé par Claude Haiku d'Anthropic. À chaque question posée, il analyse l'ensemble des documents répliqués sur SILEX et retourne une réponse précise, ancrée dans le contenu réel des fichiers, avec citation systématique de la source. Testé en conditions réelles, les résultats sont fiables et directement exploitables.



Démonstration de la plateforme SILEX

Une vidéo de **10 minutes** montre l'upload d'un document, la réplication simultanée, les métriques en temps réel, l'interaction avec Ody, la suppression coordonnée et les tests de résilience du système.



Fonctionnalités livrées et validées

14 fonctionnalités livrées

4 providers actifs

4× réplication par fichier

100% objectifs atteints

Réplication Multi-Cloud

- ✓ Upload drag & drop — PDF, image, Word
- ✓ Fan-out simultané vers 4 destinations
- ✓ Statut en temps réel par provider
- ✓ Suppression coordonnée sur tous les providers

Backend & Infrastructure

- ✓ API REST FastAPI + PostgreSQL (Docker)
- ✓ Architecture modulaire : base.py + 4 providers
- ✓ Variables d'environnement sécurisées (.env)

Intelligence Artificielle / ChatBot

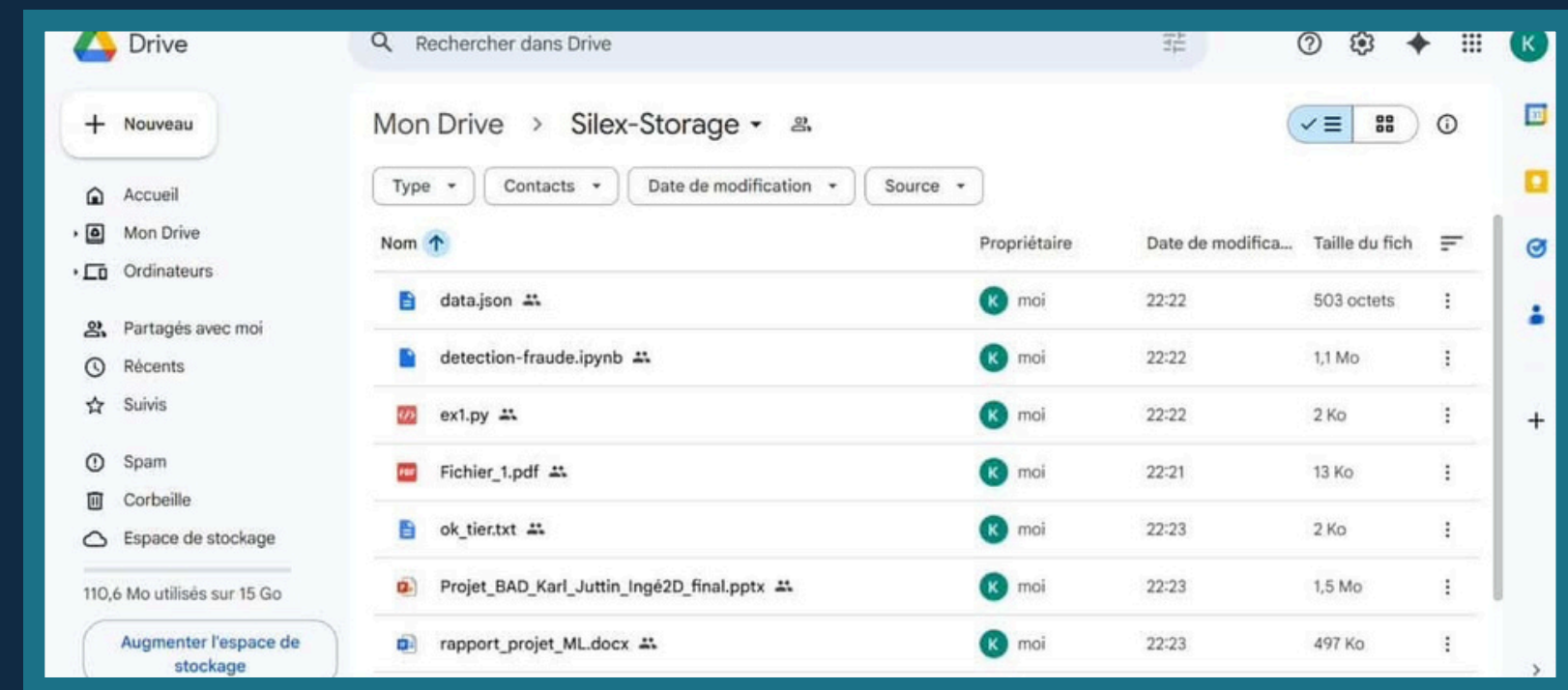
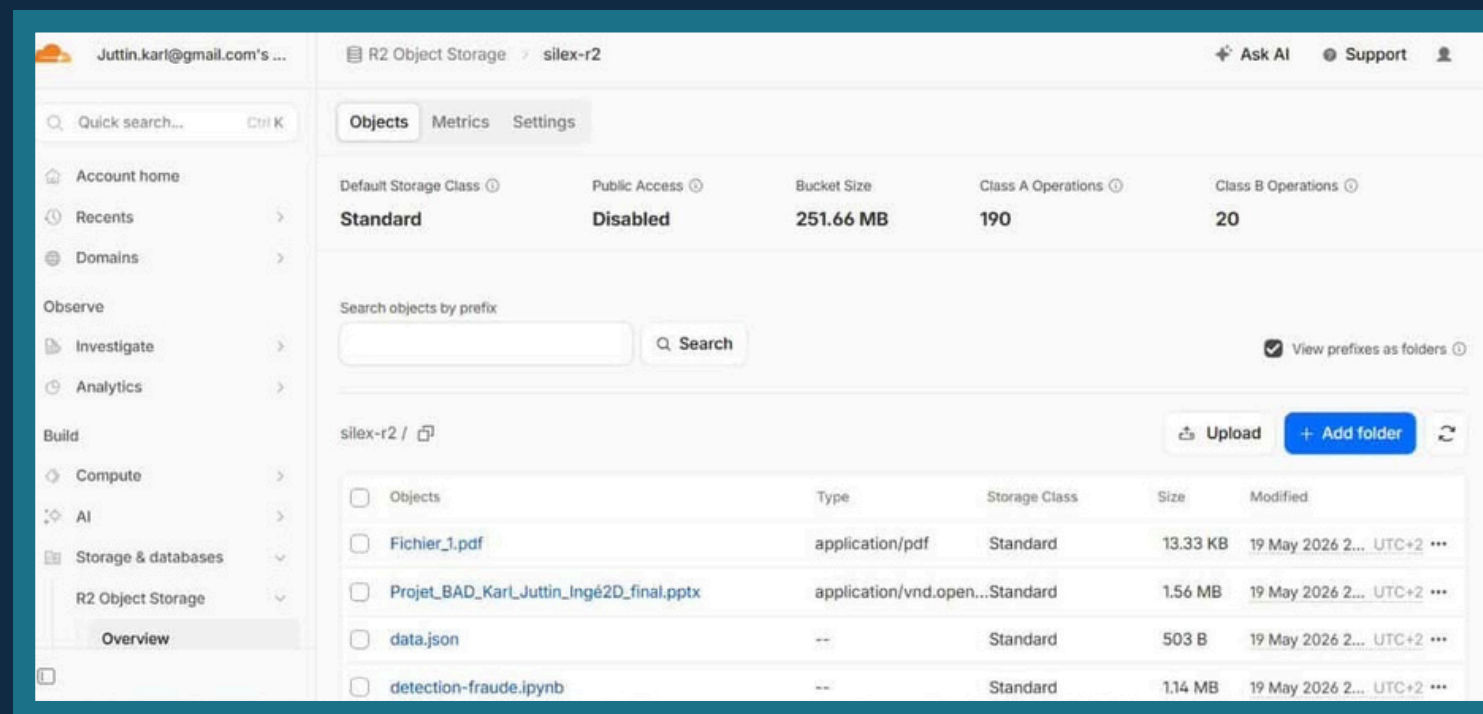
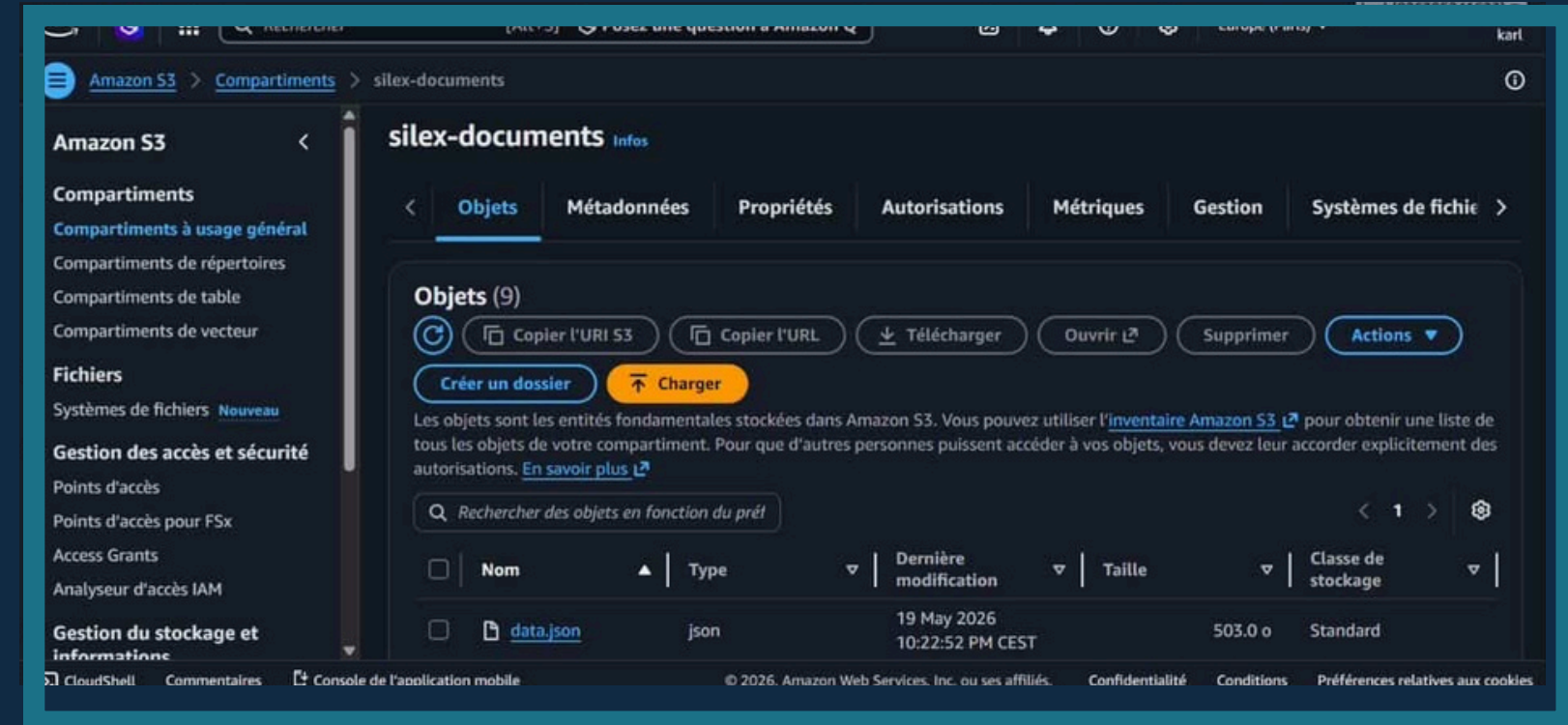
- ✓ Extraction de texte via PyMuPDF
- ✓ Assistant Ody avec Claude Haiku
- ✓ Réponses ancrées dans le contenu des fichiers
- ✓ Citation systématique de la source

Tests & Validation

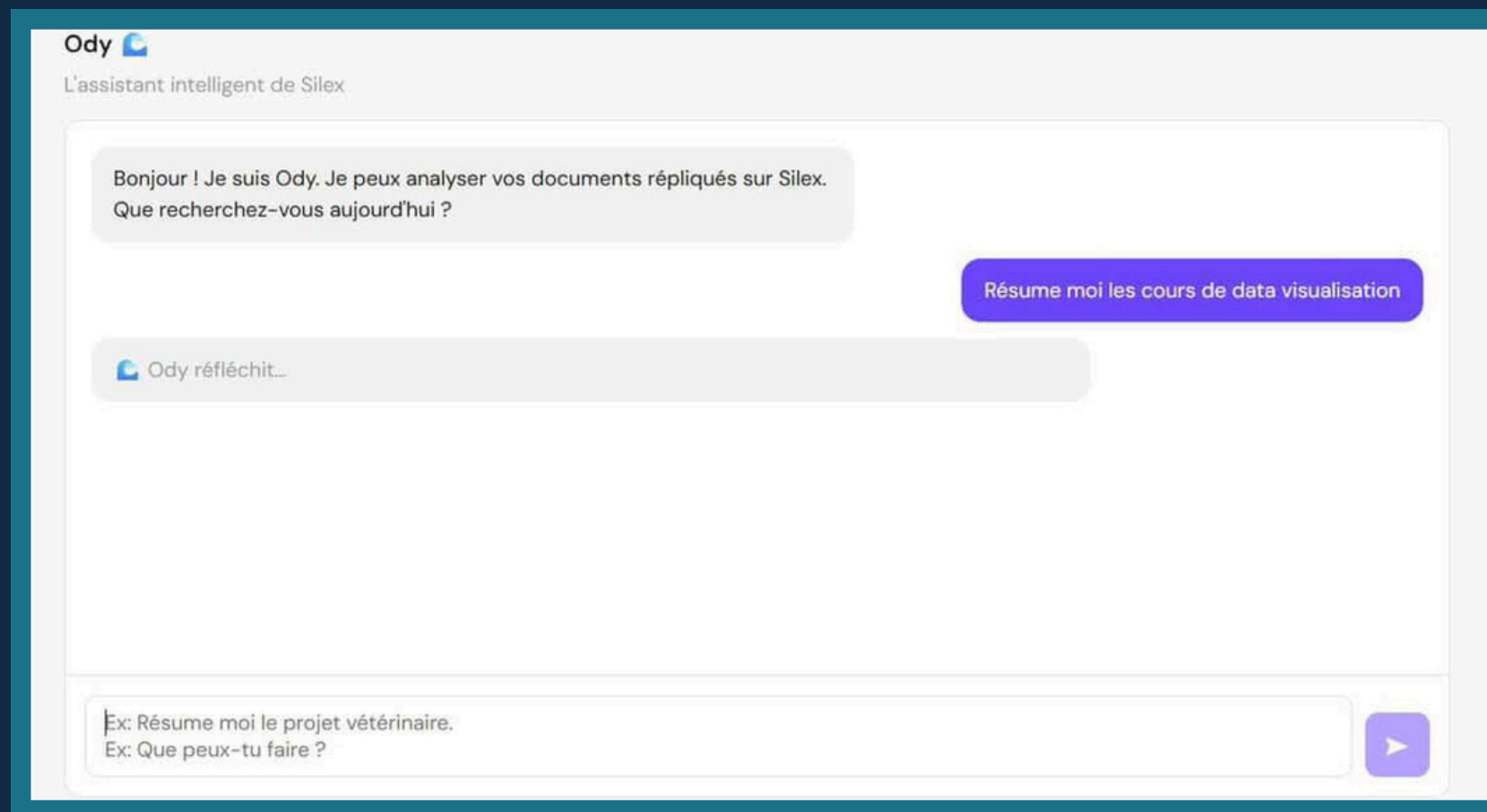
- ✓ Tests de réplication en conditions réelles
- ✓ Résilience : failure partielle tolérée
- ✓ Démonstration vidéo 10 min complète

Limites et Perspectives

- Donner une interface qui permette de modifier les paramètres pour changer de compte.
- Créer une application mobile.
- Ajouter plus de drives disponibles.
- Limite des plateformes qui bloquent les accès.



Conclusion



SILEX répond aux deux problématiques initiales : la résilience du stockage via la réplication automatique vers quatre destinations, et l'accessibilité intelligente via Ody.

Le projet est fait de manière à gérer une bonne scalabilité afin d'anticiper les futurs providers.

github.com/Artemasu/replication-silex